The Birth of MacWave[1]

It began with a sudden impulse.

On 2026-08-05, I was looking at Homebrew[2] and MacPorts[3], at my jailbroken iPhone[4] and at the Mac with theos[5], ldid(2.1.5-procursus)[6], TrollRestore[7] and Choma[8] installed, and I could not help wondering: why is there no package manager built specifically for jailbreak developers? (Author's note: back then I did not know that Homebrew already carried theos and ldid-procursus.)

So I decided to become that "Builder", and with that, this "great work" raised its curtain!

That day I logged into GitHub, which I had not used for a long time, and created a project called "MacWave".

Why this name? Honestly, it was arbitrary. At first I only planned to support the Mac, so I wanted a Mac- prefix (mirroring MacPorts), and I picked a name at random.

That same day I wrote the smallest prototype with the DeepSeek web client (VS Code was not set up yet), chose the stack I knew best, Python (3.14.8), and ran it locally (see the terminal records below).

1. Installing 1.0.0 and installing packages with `wave install`

```
Last login: Wed Aug  5 21:11:31 on ttys002
gaochengfang@gaochengfangdeMacBook-Air ~ % curl -fsSL https://raw.githubusercontent.com/Sha0huaZhang/MacWave/main/install.sh | bash
🌊 Welcome to MacWave!
🌊 Installing to /Users/gaochengfang/.local/macwave/bin...
🌊 Downloading wave.py...
🌊 Checking Python dependencies...
🌊 'requests' library is already installed.
🌊 Adding MacWave to PATH in /Users/gaochengfang/.zshrc...

🌊 Installation complete!
🌊 Try it now: wave install test_001
🌊 To apply PATH changes, run: source /Users/gaochengfang/.zshrc
gaochengfang@gaochengfangdeMacBook-Air ~ % source ~/.zshrc
gaochengfang@gaochengfangdeMacBook-Air ~ % wave -V
MacWave 1.0.0 🌊
gaochengfang@gaochengfangdeMacBook-Air ~ % wave install test_001
🌊 Error: Failed to fetch repository data:
HTTPSConnectionPool(host='raw.githubusercontent.com', port=443): Read timed out. (read timeout=10)
gaochengfang@gaochengfangdeMacBook-Air ~ % wave install test_001
🌊 Downloading test_001...
. 🌊
🌊 Successfully installed test_001 to /Users/gaochengfang/.local/macwave/bin/test_001
🌊 Ready to ride! You can now run: test_001
gaochengfang@gaochengfangdeMacBook-Air ~ % wave info test_001
🌊 Name:        test_001
🌊 Version:     1.0.0
```

```
🌊 Author:      Sha0huaZhang
🌊 Description: MacWave test package.
gaochengfang@gaochengfangdeMacBook-Air ~ % wave install machox
🌊 Downloading machox...
..........................  🌊
🌊 Successfully installed machox to /Users/gaochengfang/.local/macwave/bin/machox
🌊 Ready to ride! You can now run: machox
gaochengfang@gaochengfangdeMacBook-Air ~ % ls -la ~/.local/macwave/bin
total 432
drwxr-xr-x  5 gaochengfang  staff     160  8  5 21:25 .
drwxr-xr-x  4 gaochengfang  staff     128  8  5 21:24 ..
-rwxr-xr-x  1 gaochengfang  staff  194080  8  5 21:25 machox
-rwxr-xr-x  1 gaochengfang  staff      44  8  5 21:24 test_001
-rwxr-xr-x  1 gaochengfang  staff   17848  8  5 21:23 wave
gaochengfang@gaochengfangdeMacBook-Air ~ %
```

2. Output of `wave list`, `wave search` and `wave info`

```
Last login: Wed Aug  5 21:38:59 on ttys002
gaochengfang@gaochengfangdeMacBook-Air ~ % wave list
🌊 Installed packages:
  - test_001 (installed at /Users/gaochengfang)
  - machox (installed at /Users/gaochengfang)
gaochengfang@gaochengfangdeMacBook-Air ~ % wave search core
🌊 Found 1 package(s) matching 'core':
  - machox: CoreTrust bypass for iOS 16.7 (20H19) & 17.0.1
gaochengfang@gaochengfangdeMacBook-Air ~ % wave search test
🌊 Found 1 package(s) matching 'test':
  - test_001: MacWave test package.
gaochengfang@gaochengfangdeMacBook-Air ~ % wave search notfound
🌊 No packages found matching 'notfound'
gaochengfang@gaochengfangdeMacBook-Air ~ % wave info machox
🌊 Name:        machox
🌊 Version:     1.0.0
🌊 Author:      Sha0huaZhang
🌊 Description: CoreTrust bypass for iOS 16.7 (20H19) & 17.0.1
🌊 Homepage:    https://github.com/Sha0huaZhang/MachOX
gaochengfang@gaochengfangdeMacBook-Air ~ %
```

3. `wave update`, `wave upgrade` and uninstalling

```
Last login: Wed Aug  5 21:43:35 on ttys004
gaochengfang@gaochengfangdeMacBook-Air ~ % wave update
🌊 Updating package index...
🌊 Package index updated successfully. Found 2 packages.
gaochengfang@gaochengfangdeMacBook-Air ~ % wave list
🌊 Installed packages:
  - test_001 (installed at /Users/gaochengfang)
  - machox (installed at /Users/gaochengfang)
gaochengfang@gaochengfangdeMacBook-Air ~ % wave upgrade machox
🌊 Upgrading 'machox' to version unknown...
🌊 Downloading machox...
..........................  🌊
🌊 Successfully installed machox to /Users/gaochengfang/.local/macwave/bin/machox
🌊 Ready to ride! You can now run: machox
gaochengfang@gaochengfangdeMacBook-Air ~ % wave uninstall test_001
🌊 Removed binary: /Users/gaochengfang/.local/macwave/bin/test_001
```

```
🌊 Successfully uninstalled 'test_001'.
gaochengfang@gaochengfangdeMacBook-Air ~ % wave list
🌊 Installed packages:
  - machox (installed at /Users/gaochengfang)
gaochengfang@gaochengfangdeMacBook-Air ~ %
```

I was well aware of how important a domain name is to a project, so the very next day I bought macwave.org at NameSilo[9] ($14.99/year) and added a 302 record in DNS pointing to https://github.com/Sha0huaZhang/MacWave.

At that point the goal was simply "a small program I can use myself". I did not think much about it, and I certainly never imagined the scale it would reach.

Later I frantically piled on features over a few days, but I had not taken part in designing the architecture. That later forced a rewrite, when I started designing the core architecture and writing the logic myself — but that is a story for later.

Then I learned about GitHub Pages[10] and started writing a web page, no longer relying on the 302 redirect. I wrote static HTML with the DeepSeek web client, iterated many times, designed the UI and icons, and soon had a program with decent style but poor screen adaptation. I figured "that's good enough, it's not like I'm building a rocket", and only optimized it much later.

By then the site already had almost every feature: latest version, latest beta (later removed), introduction, commands, arguments, contact details…

I had my first custom email address: hi@macwave.org.

Next I added a colorful progress bar and help content, so the terminal was no longer dull; added mandatory SHA256[11] verification to make downloads safe; added `-C`, `--continue` and `.partial` so downloads no longer had to worry about network interruptions; added prompts at risky points (in bold red type) to lower the chance of mistakes; made downloads go to `/tmp` first so that incomplete or dangerous packages would not pollute anything else; and refined many prompts to be friendlier…

At last, on 2026-08-16, after many iterations, betas and RCs, MacWave reached its first stable release — 1.0!

If you look closely, you will notice a small change: the download command on the website went from `curl | bash` to `/bin/bash -c "$(curl …)"`. This is designed for weak networks (like mine), so that a streamed script does not end up half-downloaded when the connection drops. The absolute path is used to avoid PATH[12] pollution.

One important detail here: because this was designed for jailbreaking, precise version numbers mattered. So packages were all placed in `~/.local/macwave/bin/{package}@{version}`, which started multi-version isolation.

But I gradually noticed that as the code grew, the single main program became bloated: `wave.py` went from just over 300 lines straight to over 1200. Not only was every edit painful, it also carried patches — and those should not pollute the main program. So I began modularising (my own design),

but there was a lot involved, so I decided to validate with a minimal step first. In 1.1, the downloader `pkginstaller.py` was split out first.

Entering 2.0, large-scale decoupling and modularisation began. During this period I did a great deal of bold technical experimentation and trial and error, and fortunately found a decent approach in the end.

First, I found the arch field's `arm64`, `x86_64` and `Any` too confusing, so I split them into `pkginfo_arm64` and `pkginfo_amd64` — the prototype of today's architecture split in the package index.

Because 1.0 used `repo.json`. That format is very hard to read and nested braces are error-prone, and it was not friendly to MacWave's package index either. So I tried several approaches. At first I used a DSL[28] and `parser.rb` to parse them. That went through several iterations, but the syntax was complex and the parser produced many errors and pitfalls, until I found a better approach:

One day I thought: the index has a lot of repeated fields (package name, author, …). Could they be used only once? Oh! Just make it a directory!

So I created the `infosource` branch to hold the package index. Under https://raw.githubusercontent.com/Sha0huaZhang/MacWave/infosource/pkginfo_arm64 (or amd64)/package-name there are a few files: `_package-name@version` (several entries) and `_package-name@common`. The `_package-name@version` files record things that may change, such as the package URL and SHA256; `_package-name@common` records what does not change: author, homepage, … This way the package index can be released on its own (no need for users to update anything; the program pulls it automatically), and an independent Index Update was introduced.

Also, I stopped using a local index — a local index is really a nuisance: users are forced to download unnecessary content, and they have to update it manually (many forget, like me) or they cannot get new versions. And developers had to ship index updates as well… So I switched to pulling it live every time. One argument against that is that users need a network connection to look up package information, but for anyone who wants to download a package, a network connection is a given, so it is not a problem.

Later I realized that many operations are easier in Shell than in Python — unzipping, moving, computing SHA256… takes a single command in Shell. So I decoupled many `.sh` files and kept the parts that Python handles more conveniently as `.py` files. The directory structure and files from then on were very similar to today's MacWave. I wrote down a few basic principles at the time: `lib/` holds the core programs, `pkg/` holds programs related to packages.

It takes only a few sentences to describe, but 2.0 went through a great deal of failure and trial and error. Along the way the AI-written code logic became chaotic and produced bugs that no amount of patching could fix. I realized that without good architecture, even the cleverest code is hard to maintain. So I made a historic and extremely important decision — rewrite all of MacWave. That decision determined MacWave's present and its future.

I removed argument parsing from the main program and made it purely a "dispatch entry": a mapping table and a dictionary recognize the user's command and dispatch it, as the original string, to

each program, letting each program parse it itself. That way the main program is only 80 lines and very clean.

I redesigned every download flow and every step, drew the architecture diagram, wrote the core logic, and made sure the code stayed under my control.

I started trying VS Code with the DeepSeek API[13], gaining an advanced toolchain that also laid the foundation for MacWave's future. I also added a rule: when starting a new minor version, create the version-number branch at the same time, keep it in sync with main, until the next minor version is released.

At the same time I began using local Git[14] repositories instead of working purely in the browser. That was not only personal technical progress but a necessary step for MacWave's sustainability.

In the end, 2.0 was skipped and 2.1 was released. That version had no functional updates and even cut some non-core features — purely to streamline the architecture and make future maintenance easier.

I also found an earlier oversight: I had forgotten to add the packages to PATH! That meant users had to use absolute paths (`~/.local/macwave/bin/package/version`) — painful! So I fixed it immediately.

I thought: with such a refined design, how could it serve only jailbreak development? So I quietly changed "jailbreak developers" to "software developers", and MacWave's goal shifted to being a general-purpose package manager. I also quietly changed the website's "latest beta" to "latest version release date", because users did not seem to need betas.

Since it is a general-purpose package manager, users should have more choices instead of being limited to a specific path (such as `/usr/local`[15], `/opt/homebrew`[16], `/opt/local`[17], `/sw`[18], `/opt/sw`[19], …). The options I offered were: 1. `~/.local/macwave`, 2. `/opt/macwave`, 3. `/usr/local/macwave` (Intel Mac only)[20], 4. custom. To keep the program from failing to find the directory and to record more information, I put `config.json` (the config file, recording paths and so on) and `VERSION.json` (recording version information) in `/opt/macwave_config`.

Previously wave itself was treated as a package, like brew and port, under `bin/` (or the equivalent directory), so I had added a "protected package": wave. You could not `wave uninstall wave`, or it would say "protected package". But in MacWave 2.0 I thought of a better approach: put them in `lib/` and make wave a symlink. That way wave can still be invoked, the structure is more reasonable, and there is no need to introduce a "protected package" (the feature was removed in 2.0).

In MacWave 2.0 I also thought that Linux should be supported in the future (Homebrew and Linuxbrew[21] do the same), so I reserved space: the LinuxWave repository and the site linuxwave.macwave.org (later 301-redirected to the new address linuxwave.org when the website was updated). Those reservations were only activated in 2.5.

Then I began designing 2.2, introducing dependency handling. Naturally, dependencies followed the same design as packages, strictly separating version numbers. But a hard question appeared: how do you record a package's dependencies and its dependencies' dependencies? (Otherwise uninstalling

leaves debris, or you end up in “dependency hell”.) Traditional software (such as MacPorts) usually records them in a database, but that is error-prone and easy to lose. Is there a better way?

Remember, I used to play with TrollStore[22], and I had used TrollStore to install Filza[23]. I saw that TrollStore places a hidden marker file (empty) `.TrollStorePersistenceHelper` in the directory of its persistence helper (TrollStore PersistenceHelper[24]), and a marker file (empty) `_TrollStore` under apps installed by TrollStore. There was the answer!

After a lot of design work, consideration and refinement, the final scheme was: create a `_DEPS` file in the package’s directory recording which dependencies and versions the package uses, one per line, wrapped in straight quotes `""`; and in the dependency’s directory create an empty file `.depped_package@version` recording which packages depend on it. Of course some dependencies have dependencies of their own; same idea. This essentially blurs the distinction between a package and a dependency — and indeed, in essence they are all executables.

But packages and dependencies do have a fundamental difference, so I chose `.depped_pkg_package@version` and `.depped_dep_dependency@version` to tell them apart.

```
Last login: Sat Oct 24 23:50:54 on ttys011
gaochengfang@gaochengfangdeMacBook-Air ~ % cd ~/.local/macwave/bin
ls -d */ | head -n 5
fd@10.5.0/
fileicon@0.3.5/
fzf@0.74.4/
htop@3.5.3/
jq@1.8.2/
gaochengfang@gaochengfangdeMacBook-Air bin % cat wget@1.25.0/_DEPS
"gettext@0.21.0"
"libiconv@1.16"
"libidn2@2.3.8"
"libunistring@1.3"
"openssl@3.0.15"
"pcre2@10.42"
"zlib@1.2.13"
gaochengfang@gaochengfangdeMacBook-Air bin %
# enter openssl's dependency directory
cd ~/.local/macwave/deps/openssl/openssl@3.0.15/

# see whether any hidden marker files are inside
ls -la | grep "depped"

-rw-r--r--   1 gaochengfang  staff     0  9 23 20:46 .depped_dep_libevent@2.1.13
-rw-r--r--@  1 gaochengfang  staff     0 10 12 16:36 .depped_pkg_wget@1.25.0
gaochengfang@gaochengfangdeMacBook-Air openssl@3.0.15 %
```

How does a package find its dependencies? After weighing the options, I chose to modify Mach-O[26] addressing with Xcode Command Line Tools[25].

Along the way I also designed a large family of commands letting developers/users manage and replace dependencies by hand. But then I thought: the most important thing about a tool for developers/users is peace of mind. They come home from work exhausted — who has the energy to handle dependencies one by one? So I introduced automatic dependency management: when downloading a package, first check whether the required dependencies and versions exist; if they do, record them; if not, download and then record. When uninstalling a package, check whether that package’s

dependencies are still required by other packages after this removal. If so, do not delete, only record. Otherwise delete and record. Dependencies of dependencies are handled recursively.

The advantages of this scheme: 1. Atomicity. Because the marker files are empty, there are only "success"/"failure" and "recorded"/"not recorded" — no intermediate state, so the awkward "half-recorded" case cannot happen. 2. Low error rate. Empty files are very convenient: downloading only needs to create a file rather than write one, and uninstalling only needs to delete or check whether a file exists rather than read it. Far less prone to mistakes. 3. Easy migration: you will not lose dependency records because you forgot to migrate a database.

And so — MacWave 2.2 was released! MacWave became a real package manager from then on!

Having come this far, an incompatible and unattractive website clearly could not satisfy me, so I rewrote the site into what it looks like today, compatible with all screen sizes and ratios.

But even at 2.2, MacWave still had no self-update command, so in 2.3 I added `selfupdate`. Why not `update`? Because many people cannot tell update and upgrade apart (me included), and `selfupdate` states plainly that it updates itself, while also mirroring MacPorts — a good choice. `selfupdate` has gone through many iterations since, and now supports fetching some migration commands remotely instead of hard-coding them, preventing `selfupdate` from failing en masse when the architecture changes. `selfupdate` gets its data from the `configdata` branch

Then a small interlude — I found that `wave -h` still had commands left over from 1.x, so I fixed it in 2.3.1.

I also noticed: `wave -h` prints all the help information, but if a user is only unfamiliar with the usage of one command, isn't that a nuisance? So I added `wave command -h`, which shows the usage help for that command, examples included.

I thought: although forcing version numbers in PATH (`package@version`) lets users know and reminds them which version they are using, it is inconvenient day to day, and sometimes users just want to use a tool without caring which version it is. So in MacWave 2.4 I added the link mechanism and a series of automatic handling. The rules are as follows:

Since MacWave 2.4, once a symlink without a version number is created, MacWave automatically maintains the version it points to:

1. When the highest version is uninstalled, the versionless symlink points to the highest remaining version, and the uninstalled version's own symlink is removed as well.
2. Installation points the versionless symlink at the version being installed; adding `--unlink` to this installation skips that step.
3. When a package no longer has any version, its versionless symlink is removed.
4. Packages that were manually unlinked are remembered; later uninstalls will not create links for them again.
5. In all cases, the name with the version number remains callable, even when a versionless symlink exists.

There is also the following command family:

```
wave link {pkg}
wave linkquery {pkg}
wave unlink {pkg}
wave link -a, --all
```

```
wave install {pkg} --unlink
```

Then I set up a “link everything” command for previously installed packages: `wave link -a` (or `--all`)

Then I reduced the website’s command to a single line: I appended `&& source ~/.zshrc` to the install command so nobody forgets.

Once, while uninstalling a package, I found that asking for the version number is very convenient when there are multiple versions, but when there is only one it seems redundant, so in 2.4.2 I made `wave uninstall` stop asking for the version when only one exists.

I thought: if several users all use a per-user install, that should be legitimate, but the configuration was all written to `/opt/macwave_config` and would overwrite each other — not appropriate. So in 2.5 I changed the location of the config file. The rules are as follows:

The config file is no longer fixed in `/opt/macwave_config`; it follows the install location:

- Installing to a directory that needs sudo (`/opt/macwave`, `/usr/local/macwave`, a custom directory outside the home directory): config in `/opt/macwave_config`
- Installing to a directory that does not need sudo (`~/.local/macwave`, a custom directory inside the home directory): config in `~/.config/macwave_config`

When reading, the system-level one takes priority: as long as `/opt/macwave_config/config.json` exists it is used, otherwise it falls back to `~/.config/macwave_config`. Automatic migration was also implemented in `install.sh` and `selfupdate`

In this version I ported MacWave to Linux, and the long-reserved LinuxWave was finally activated. Mach-O addressing was ported from Xcode Command Line Tools to ELF[27], and the path options became: 1. `~/.local/linuxwave`, 2. `/opt/linuxwave`, 3. `/usr/local/linuxwave`, 4. `/home/linuxwave/.linuxwave`, 5. custom. The config file locations are `/etc/linuxwave_config` and `~/.config/linuxwave_config` respectively; shared installs (option 4) always use `/etc/linuxwave_config`, to ensure that every user on the machine resolves to the same installation. It needs `patchelf`, Python 3.14+, and a one-time sudo; the linuxwave account is created automatically if it does not exist. After a shared install, the user is prompted to log in again or run `newgrp linuxwave`. Here I borrowed Linuxbrew’s user-group approach (thanks Linuxbrew, though I still hit plenty of pitfalls), which gave us path 4 (`/home/linuxwave/.linuxwave`). That really is a good scheme: not only globally shared, but installing packages needs no sudo either.

Next I created the strictest CI tests ever: randomized — all kinds of commands and arguments, wrong and right, randomly interrupted, and so on. Anything you can think of could happen in the CI tests. After extensive testing and optimization, the CI is now very stable.

I thought: what if a user uses my `install.sh` inside a script? Nobody in a script is going to answer Yy. So in MacWave 2.5 I added automated installation:

Install

Option	Description
`--silent, -S`	No interaction at all: the directory menu and the license agreement are answered automatically
`--dir-option=N`	Choose menu item N directly (no menu is shown)
`--dir-option=N=DIR`	Choose menu item N; for the custom item, use DIR as the install directory
`--help`	List all options

Example:

```
# script is already local
bash install.sh --silent --dir-option=1

# install straight from the repository
curl -fsSL https://raw.githubusercontent.com/MacWaveOrg/MacWave/HEAD/lib/install.sh |
bash -s -- --silent --dir-option=2
```

Uninstall

Option	Description
`--force`	Skip confirmation; delete MacWave and its configuration
`--remove-user`	Only for argument compatibility with LinuxWave; MacWave has no dedicated account, so this does nothing
`--help`	List all options

But I felt there were some problems: 1. Since option 3 (Intel) or option 4 (Apple Silicon) are custom directories, why write a number at all? It is also error-prone. 2. The command above is still `curl | bash`; it should be changed to the more robust `/bin/bash -c "$(curl …)"`. So for these two points I released MacWave/LinuxWave 3.0.1.

In MacWave/LinuxWave 2.6 I realized that MacWave/LinuxWave had become a huge project, so in 2.6, just as mxcl[2] handed the Homebrew repository over to the Homebrew organization, I moved MacWave/LinuxWave from my personal account to the MacWaveOrg/LinuxWaveOrg organization, and bought linuxwave.org (the old domain linuxwave.macwave.org was 301-redirected to the new one), dedicating one organization to a single project. It was an unprecedented migration: to make maintenance easier, some branches were migrated into repositories. Read-only snapshots of the old repositories were copied and archived; a week later I added a legacy- prefix to the names of those branches.

Thanks to the purchase of the new domain linuxwave.org, LinuxWave released version 2.6.2 specifically.

I thought: since there is already a dedicated organization, `infosource`, `configdata` and `gh-pages` could perfectly well become repositories, which is even more convenient. So I released MacWave/LinuxWave 3.0, giving users of old versions a 7-day migration window.

Between MacWave/LinuxWave 2.6 and 3.0 the following major migrations took place:

Item	Before	After	Version	Note
MacWave repository	https://github.com/Sha0huaZhang/MacWave	https://github.com/MacWaveOrg/MacWave	MacWave2.6	Repository address migration
LinuxWave repository	https://github.com/Sha0huaZhang/LinuxWave	https://github.com/LinuxWaveOrg/LinuxWave	LinuxWave2.6	Repository address migration
MacWave old repository read-only snapshot		https://github.com/MacWaveOrg/legacy-MacWave	MacWave2.6	Read-only repository snapshot
LinuxWave old repository read-only snapshot		https://github.com/MacWaveOrg/legacy-LinuxWave	LinuxWave2.6	Read-only repository snapshot
MacWave/infosource branch	https://github.com/Sha0huaZhang/MacWave/tree/infosource	https://github.com/MacWaveOrg/infosource	MacWave3.0	Branch split into a repository, later renamed legacy-infosource
LinuxWave/ infosource branch	https://github.com/Sha0huaZhang/LinuxWave/tree/infosource	https://github.com/LinuxWaveOrg/infosource	LinuxWave3.0	Branch split into a repository, later renamed legacy-infosource
MacWave/gh-pages branch	https://github.com/Sha0huaZhang/MacWave/tree/gh-pages	https://github.com/MacWaveOrg/Pages	MacWave3.0	Branch split into a repository
LinuxWave/gh-pages branch	https://github.com/Sha0huaZhang/LinuxWave/tree/gh-pages	https://github.com/LinuxWaveOrg/Pages	LinuxWave3.0	Branch split into a repository
MacWave/configdata branch	https://github.com/Sha0huaZhang/MacWave/tree/configdata	https://github.com/MacWaveOrg/configdata	MacWave3.0	Branch split into a repository, later renamed legacy-configdata
LinuxWave/ configdata branch	https://github.com/Sha0huaZhang/LinuxWave/tree/configdata	https://github.com/LinuxWaveOrg/configdata	LinuxWave3.0	Branch split into a repository, later renamed legacy-configdata
LinuxWave website	linuxwave.macwave.org	linuxwave.org	LinuxWave2.6.2	Website moved to its own domain

In MacWave 3.0.2 I made a small convenience change: `--ver` can now join the version number with = (previously only a space worked), so installing a specific version now has three forms. Using ffmpeg as an example:

```
wave install ffmpeg@9.0
wave install ffmpeg --ver 9.0
wave install ffmpeg --ver=9.0
```

Later, Python 3.15 was released. To try the new features while staying compatible with newer versions, I upgraded my Python (3.14.8 >>> 3.15.0), and MacWave's linkquery broke! It reported a missing request library. I found that MacWave's shebang is `#!/usr/bin/env python3`; env now pointed

at 3.15, but request was installed on 3.14 under pyenv. So, following how Homebrew packages Ruby, from 3.1 I bundled MacWave's own independent CPython interpreter. This not only avoids the problem of a user's Python being too new or too old (which is often inconvenient to update or downgrade), it even removes the need for users to install Python at all, which undoubtedly lowers the barrier to entry enormously.

I also noticed: at this point there was still no `wave upgrade` command. Adding an upgrade command is easy for other package managers, but in MacWave it is a question that needs careful thought. Because MacWave naturally supports multi-version isolation and coexistence, what does upgrade even mean? Download the latest version, upgrade the existing version, delete the old version and download a new one, or something else? In the end, I settled on the following scheme:

```
wave upgrade {pkg}                      # latest version, keep old version
wave upgrade {pkg} --remove-old-version  # latest version, remove old version
wave upgrade {pkg} --remove-old-versions # same as above, completely equivalent
wave upgrade {pkg} --rov                 # same as above, completely equivalent
wave upgrade {pkg} -X                    # same as above, completely equivalent
```

At first I wanted to give `--remove-old-versions` a short flag, but could not find a suitable one: `-R` is usually taken and `-r` usually means recursive, so I simply used the shorter long option `--rov`.

At first only the long option `--remove-old-version` was valid, but over time even I could not remember whether it was verison or versions, so in 3.2.1 I made them all valid.

But I still wanted a real short flag — it really is convenient — so in 3.2.1 I could not resist making `-X` (uppercase) a short flag. Why the letter X? Because Rr needs to reserve recursion (recursive), reversal (reverse) and rollback (rollback), and the letter X not only leaves those reservations untouched but also has no fixed meaning, and I think X looks very much like the symbol for "delete" (a cross), which makes it a kind of pictogram that is easy to remember. I added it in 3.2.1.

Edited 2026-10-27, latest version at the time: 3.2.1

Every great tool begins with something insignificant.

Notes:

1. MacWave: a package manager developed by Sha0huaZhang on 2026-08-05, whose first stable release came out on 2026-08-16; it has since moved under the MacWave organization, with macwave.org as its website.
2. Homebrew: a package manager developed by Max Howell in 2009, characterized by downloading pre-built binary packages so that no local compilation is needed. It has since moved from a personal account to the Homebrew organization; website: brew.sh.
3. MacPorts: a package manager started jointly by several Apple employees in 2002, originally launched as part of the OpenDarwin project. Its characteristic is installing software into a separate directory, avoiding mixing files with system-provided software or with other package managers and thus reducing the risk of conflicts; website: macports.org.
4. The author's former main iPhone: an iPhone XR, white, 128GB, iOS 15.3.1 (19D52), on which TrollStore and Dopamine were used.
5. Theos: an open-source cross-platform build system for iOS jailbreak tweak development created by Dustin Howett, offering simplified code writing, compilation and .deb packaging; now community-maintained.

6. ldid: a lightweight command-line tool created by Jay Freeman (Saurik) for viewing and modifying the signing and entitlement information of iOS/macOS Mach-O binaries; now community-maintained, with the most widely used version being the one maintained by ProcursusTeam.
7. TrollRestore: a desktop installer for TrollStore built specifically for iOS 16.7 RC and 17.0, led by JJTech (@JJTech0130); it can also install TrollStore on iOS 15.2 and above.
8. ChOma: a C library created by Lars Fröder (@opa334) for crafting malicious signatures that exploit the CVE-2023-41991 signing vulnerability, mainly used with TrollStore.
9. NameSilo: a globally known ICANN-accredited domain registrar.
10. GitHub Pages: the free static site hosting service provided by GitHub, which reads files directly from a repository and publishes them as a website automatically. It is mostly used to host personal résumés, project documentation or blogs without buying a server; a free account is enough for public repositories.
11. SHA256: a cryptographic hash function that compresses data of any length into a unique 256-bit (32-byte) "fingerprint", from which it is practically impossible to recover the original data or to find two different inputs with the same fingerprint.
12. PATH: the environment variable the operating system uses to find executables; it determines which directories the system searches for a program when a command is typed at the command line.
13. The DeepSeek API extension for VS Code: inexpensive and easy to use.
14. Local Git: the Git version-control system running on your own computer, used to track code changes and manage commit history; commits, branches and rollbacks all work without a network connection.
15. /usr/local: the install path of Homebrew on Intel Macs.
16. /opt/homebrew: the install path of Homebrew on Apple Silicon Macs.
17. /opt/local: the install path of MacPorts.
18. /sw: the install path of Fink.
19. /opt/sw: the install path of Fink since macOS Catalina 10.15, when the system root (/) became read-only and /sw could no longer be created.
20. /usr/local: Apple started writing files to /usr/local on Apple Silicon Macs (/usr/local is no longer empty), while writing to this directory is discouraged.
21. Linuxbrew: Homebrew did not originally support Linux, and the community ported it as Linuxbrew in 2012. The core program was merged back into Homebrew in Homebrew 2.0.0 in 2019, and Homebrew-core completed the full merge in 2021.
22. TrollStore: a "permanent signing" tool on iOS that installs apps not signed by the App Store, exploiting a signing vulnerability to keep apps valid long-term; it works on iOS 14.0 beta 2 – iOS 16.7 RC/17.0.
23. Filza: a file manager commonly used in jailbreak/TrollStore environments on iOS, able to read and write system files (writable only with a rootful jailbreak).
24. TrollStore PersistenceHelper: TrollStore's "persistence helper", used to keep TrollStore and the apps it installed available after a system update or reboot.
25. Xcode Command Line Tools: the command-line developer toolset provided by Apple, including the compiler, codesign, install_name_tool and more; in this article it is used to modify Mach-O dependency addressing.
26. Mach-O: the executable format of macOS/iOS, recording binary code, dependency library paths, signatures and other information.
27. ELF: the executable format of Linux, the counterpart of macOS's Mach-O; here it refers to the binary format that LinuxWave uses for dependency addressing on Linux.
28. DSL: domain-specific language; here, a small purpose-built syntax designed for describing the package index.